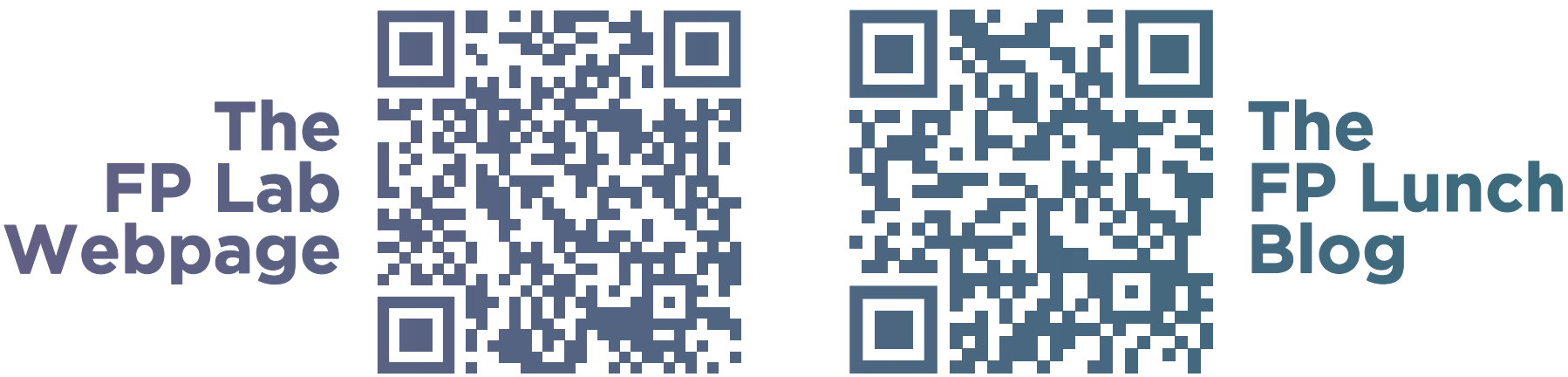




About Us

- ▶ Leading centre for programming language research for nearly 30 years
- ▶ Produced over 35 PhD graduates
- ▶ Awarded over £4m in funding in the past five years from: EPSRC, ERC, Royal Society, US Air Force



- ▶ Industry partners: Meta, Microsoft, Standard Chartered, Huawei, G-Research, Protocol Labs

Motivation

- ▶ Bugs in software are dangerous & costly
- ▶ Fundamental question: how do we **reason** about software?

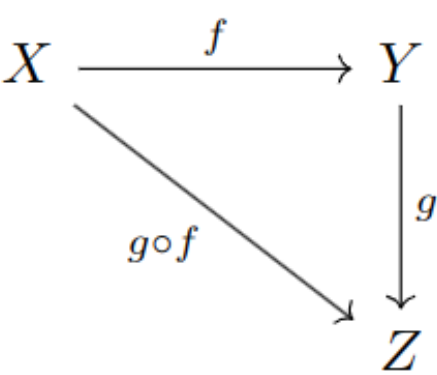
Solution

- ▶ Model programs mathematically and **prove** them correct

Compositionality

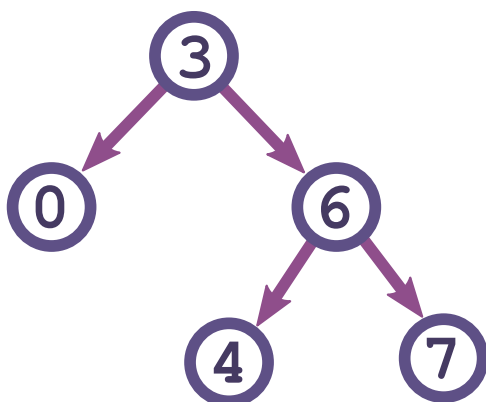
Key Idea

Reason about a large program by reasoning about smaller sub-programs



Category Theory is the mathematical theory of compositionality. We use it to organise and construct computer programs and their semantics

Example Common structures in CS, such as lists, trees, etc.



[True, True, False]
[100, -6, 35]
["Hello", " world", "!"]

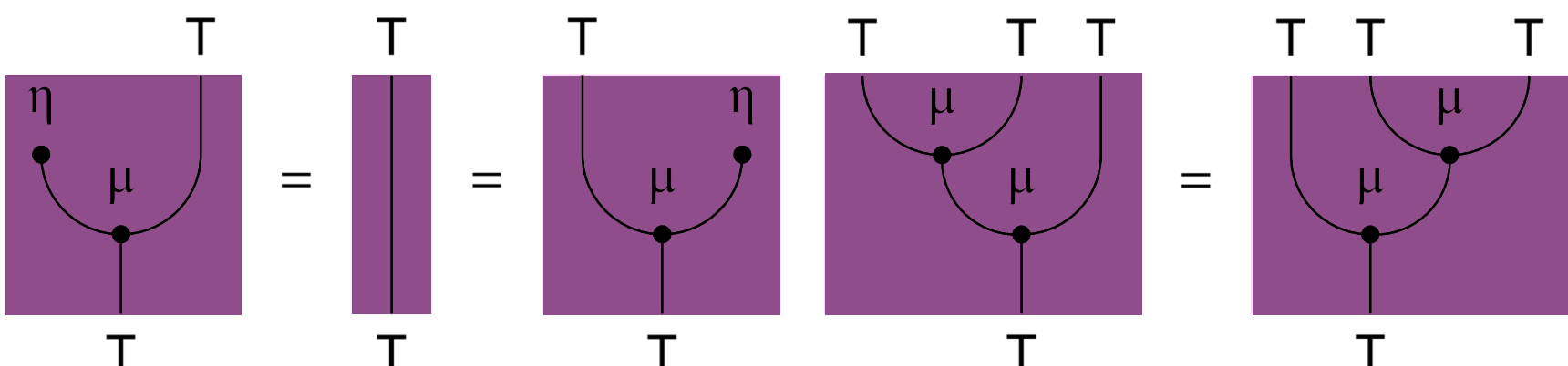
These can be **abstracted** to the category-theoretic notion of a **monad**.

Operations defined on arbitrary monads can be **instantiated** to work on any of these structures.

$$\begin{aligned} T &: \mathcal{C} \rightarrow \mathcal{C} \\ \eta &: \text{id} \Rightarrow T \\ \mu &: T^2 \Rightarrow T \\ \mu \circ T\mu &= \mu \circ \mu T \\ \mu \circ T\eta &= \mu \circ \eta T = \text{id} \end{aligned}$$



Haskell is a functional programming language, that has compositionality as a core principle of its design. It allows its users to use category-theoretic constructs like monads in their programs.



Proofs as Programs

Methods for developing correct software:

- Good** Write code $\leftrightarrow$ Test it (when are we done testing?)
- Better** Write code $\leftrightarrow$ Prove it correct (requires extra effort)
- Best** Write code *while* proving it correct

In **type theory**, mathematical proofs *are* computer programs!

Example A member of the type

$$\prod_{n:\mathbb{N}} \sum_{p:\mathbb{N}} \text{is-prime}(p) \times (p > n)$$

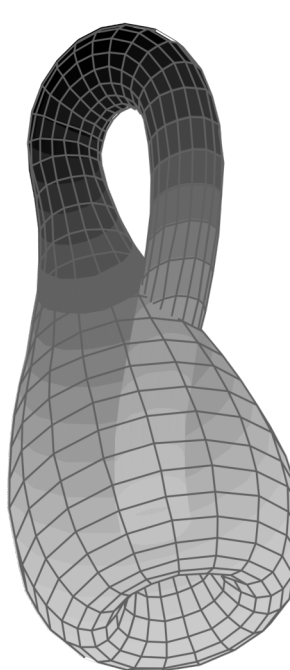
- is both:
- ▶ a **proof** that there are infinitely many primes
 - ▶ a **program** taking an input n and outputting a prime p such that p>n.



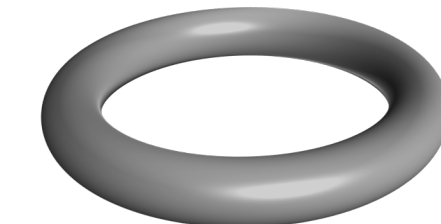
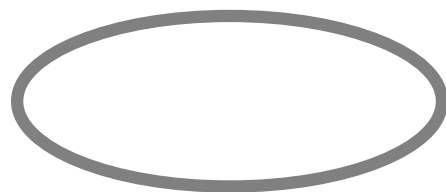
Agda is a practical implementation of type theory, that allows us to *computer-verify* proofs.

Recent research teaches us to think of types as **spaces**, and equality as **paths** in a space. This viewpoint underlies the recent development of **Homotopy Type Theory**.

For example, the following spaces are implemented as (Cubical) Agda types.



```
data S¹ : Type where
  base : S¹
  loop : base = base
```



```
data Torus : Type where
  point : Torus
  line1 : point = point
  line2 : point = point
  square : PathP (λ i → line1 i = line1 i) line2 line2
```

```
data KleinBottle : Type where
  point : KleinBottle
  line1 : point = point
  line2 : point = point
  square : PathP (λ i → line1 (~ i) = line1 i) line2 line2
```

